SOFTWARE ARCHITECTURE DESIGN DOCUMENTATION AND PATTERNS

COMP6100001 - SOFTWARE ENGINEERING
SESSION 10

UNIVERSITAS **BINA NUSANTARA**

SUBJECT MATTER EXPERT

Kenny Jingga, S.Kom., M.T.

SESSION 10 – SOFTWARE ARCHITECTURE DESIGN DOCUMENTATION AND PATTERNS

LEARNING OBJECTIVES

At the end of this session, students will be expected to be able to:

- Understand the Importance of Documenting Software Architecture
- Identify Key Components of Architectural Documentation
- Introduce Common Architectural Patterns and Their Applications
- Apply Documentation Standards and Best Practices

SESSION 10 – SOFTWARE ARCHITECTURE DESIGN DOCUMENTATION AND PATTERNS

CONTENT OUTLINE

- Documenting Software Architecture
- Architecture Description Languages (ADLs)
- Creating Architectural Views: Logical, Development, Process, Physical
- Evaluating and Validating Architecture
- Introduction to Architectural Patterns
- Applying Documentation Standards and Best Practices

DOCUMENTING SOFTWARE ARCHITECTURE

PURPOSE OF ARCHITECTURE DOCUMENTATION

- Documentation provides a blueprint for system design and serves as a communication tool between developers, stakeholders, and project managers.
- Key reasons include:
  - Ensuring design consistency across teams.
  - Providing a reference point for future maintenance and scaling.
  - Capturing architectural decisions and their justifications to support knowledge transfer.
- **Example**: An overview document for a banking app that includes system objectives, key decisions, and architectural constraints.

DOCUMENTING SOFTWARE ARCHITECTURE

TYPES OF ARCHITECTURE DOCUMENTATION AND LEVELS

- **Overview Documents**: High-level descriptions that outline system objectives, design choices, and constraints.
- **Detailed Design Documents**: Technical documents specifying components, connections, data flow, and security measures.

Documentation Levels:

- **High-Level Documentation**: For non-technical stakeholders, providing a functional overview without technical specifics.
- **Low-Level Documentation**: Focuses on code structures, algorithms, and system integration for technical team members.

ARCHITECTURE DESCRIPTION LANGUAGES (ADLS)

DEFINITION AND ROLE OF ADLS

- ADLs are specialized languages for describing system architecture, specifying components, connectors, and system configurations. They provide a formal way to capture the high-level design of software systems.

- **Examples of Common ADLs**:

  - **UML (Unified Modeling Language)**: Widely used to create visual diagrams like class, sequence, and use case diagrams.

  - **Acme**: A research-focused ADL for defining architectural styles and patterns.

- **Benefits**:

  - Facilitates early error detection and design improvements.

  - Enhances communication between design and implementation teams.

  - Supports automated code generation and design analysis.

CREATING ARCHITECTURAL VIEWS

INTRODUCTION TO ARCHITECTURAL VIEWS

- Architectural views are perspectives on the system design created to address the needs of specific stakeholders.
- Different views improve the understanding and analysis of the system's structure and function.

CREATING ARCHITECTURAL VIEWS

TYPES OF VIEWS

- **Logical View**: Illustrates the system's main functionalities, focusing on interactions among major components. **Example**: Showing user interaction with login, dashboard, and account modules.

- **Development View**: Describes the system from a developer's perspective, typically focusing on the organization of modules and layers. **Example**: Showing module dependencies for back-end and front-end teams.

- **Process View**: Highlights the runtime behavior of the system, including concurrency and communication among processes. **Example**: Tracking user actions through microservices in an e-commerce app.

- **Physical View**: Represents the deployment aspect, including the physical hardware and network topology. **Example**: Illustrating server locations for a cloud-based application to meet scalability needs.

CREATING ARCHITECTURAL VIEWS

CHOOSING RELEVANT VIEWS

- Selecting views depends on the stakeholders' needs (business, developers, IT operations), ensuring that all functional and technical aspects are covered for the audience.

EVALUATING AND VALIDATING ARCHITECTURE

IMPORTANCE OF ARCHITECTURE EVALUATION

- Regularly evaluating architecture ensures it aligns with the system's non-functional requirements, such as scalability, security, and performance.

EVALUATING AND VALIDATING ARCHITECTURE

EVALUATION TECHNIQUES

- **ATAM (Architecture Tradeoff Analysis Method)**:
  - Used to understand trade-offs in architectural decisions (e.g., security vs. performance).
  - **Process**: Identifying architectural approaches, analyzing quality attributes, and evaluating trade-offs.
  - **Example**: For a healthcare app, prioritizing security over system performance..
- **CBAM (Cost-Benefit Analysis Method)**:
  - Used to estimate the economic impact of architectural decisions.
  - **Process**: Mapping architectural components to costs and expected benefits, helping to justify design decisions based on ROI.

EVALUATING AND VALIDATING ARCHITECTURE

VALIDATION TECHNIQUES

- **Prototyping**: Developing a small-scale version to test assumptions about the architecture.
- **Simulations and Peer Reviews**: Review the architecture with experts to identify potential improvements and detect flaws early.

INTRODUCTION TO ARCHITECTURAL PATTERNS

UNDERSTANDING ARCHITECTURAL PATTERNS

- Patterns are established solutions for common design problems, helping architects ensure structural integrity and adaptability.

- **Common Patterns**:

  - **Layered (n-Tier):** Organizes the system into separate layers, each responsible for a specific type of functionality. Use case: Suitable for systems requiring clear separation of concerns, such as an online store with separate UI, business logic, and data layers.

  - **Client-Server**: Distributes work between clients (requesters) and servers (providers). Use case: Widely used in database applications and distributed systems.

  - **Microservices**: Composes an application from loosely coupled, independently deployable services. Use case: Large systems requiring scalability, flexibility, and independent deployment.

  - **Event-Driven**: Focuses on real-time responsiveness, where events trigger processing within the system. Use case: Applications requiring quick response to data changes, like a social media feed.

INTRODUCTION TO ARCHITECTURAL PATTERNS

PATTERN SELECTION AND TRADE-OFFS

- Selecting patterns is based on **project requirements**; e.g., a microservices pattern suits scalability needs, while a layered pattern works for simpler systems with distinct layers.

APPLYING DOCUMENTATION STANDARDS AND BEST PRACTICES

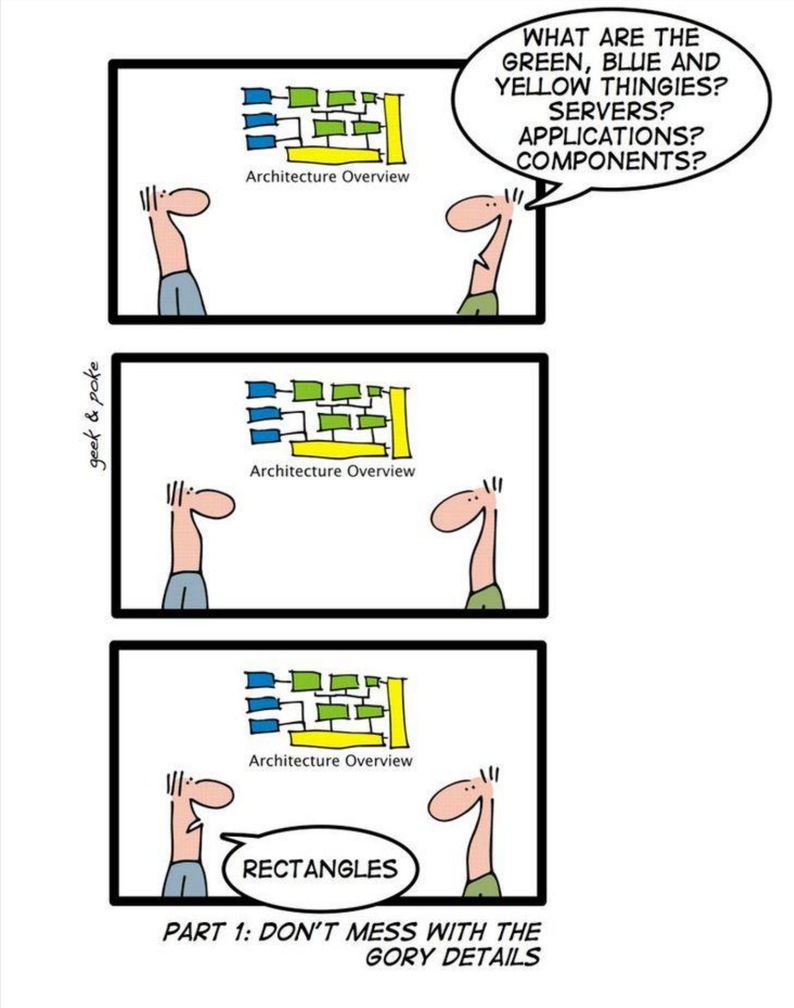
IMPORTANCE OF STANDARDS IN DOCUMENTATION

- Standards create uniformity in documentation, simplifying collaboration and ensuring that documentation remains useful over time.
- **Best Practices for Effective Documentation**:
  - **Modularity**: Organize documentation in self-contained sections for easy updates and reuse.
  - **Version Control**: Track documentation updates to maintain a history of architectural changes and decisions.
  - **Continuous Updates**: Synchronize documentation with system changes to avoid outdated or irrelevant information.

APPLYING DOCUMENTATION STANDARDS AND BEST PRACTICES

GUIDELINES FOR SELECTING A NOTATION

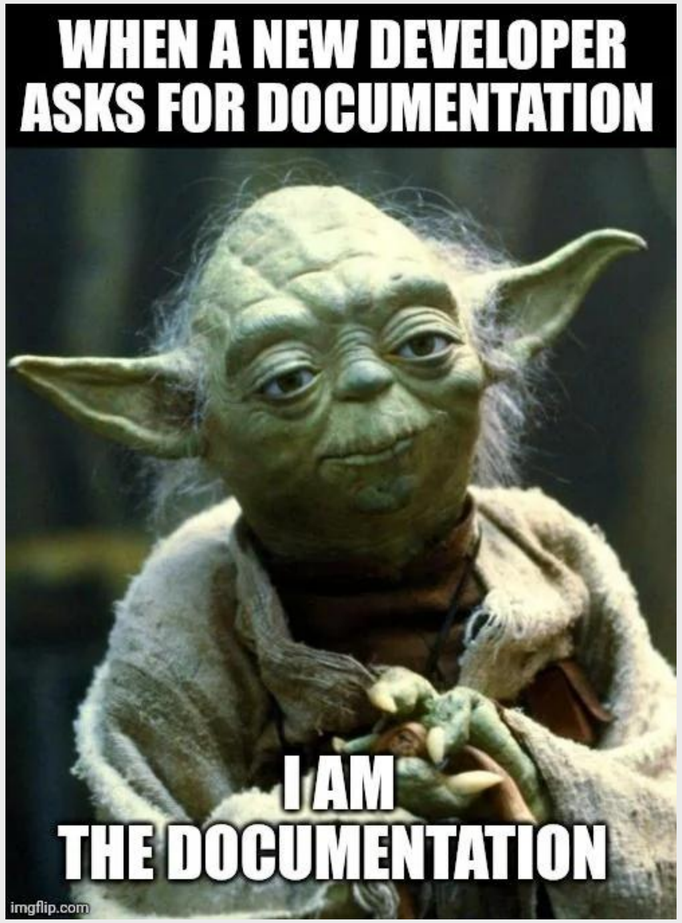
- Suitable for purpose
- Often visual for compact representation
- Usually, boxes and arrows
- UML possible (semi-formal), but possibly constraining
  - Note the different abstraction level – Subsystems or processes, not classes or objects
- Formal notations available
- Decompose diagrams hierarchically and in views
- Always include a legend
- Define precisely what the boxes mean
- Define precisely what the lines mean
- Do not try to do too much in one diagram
  - Each view of architecture should fit on a page
  - Use hierarchy



APPLYING DOCUMENTATION STANDARDS AND BEST PRACTICES

EXAMPLES OF TOOLS AND TEMPLATES

- Tools like Confluence, Google Docs, or JIRA provide templates that standardize architecture documents.
- **Example**: A Confluence template for software architecture that includes sections for high-level design, component details, and dependencies.



REFERENCES

- Titus Winters, Tom Manshreck, and Hyrum Wright. (2020). ***Software Engineering at Google: Lesson Learned from Programming Over Time.*** 1st. O'Reilly Media. California. ISBN: 978-1492082798.
- Roger Pressman and Bruce Maxim. (2020). ***Software Engineering: A Practitioner's Approach.*** 9th. McGraw Hill. New York. ISBN: 978-1259872976.
- Ian Sommerville. (2016). ***Software Engineering.*** 10th. Pearson. London. ISBN: 978-1-292-09613-1.